



Homework Assignment

Week 11: Dictionaries - Key-Value Magic

Total Points: 100 — Due: Next Class



Name: _____

Date: _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python (*using Thonny or any online IDE*)
- For setting up your code files, kindly see `instructions.pdf`
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Understanding Dictionary Structure (15 points)

Time estimate: 20 minutes

1.1 Exercise 1.1: Dictionary vs List Comparison (8 points)

For each scenario, decide whether a **List** or **Dictionary** would be better. Circle your choice and explain why in one sentence.

Scenario	List	Dict	Why?
Store cricket scores for Pakistan's last 5 matches in order	☐	☐	
Look up a student's grade using their name	☐	☐	
Track how many times each word appears in a paragraph	☐	☐	
Keep a shopping list of items to buy	☐	☐	
Store phone numbers where you look up by person's name	☐	☐	
Record the top 10 highest scores in a game	☐	☐	
Map city names to their populations	☐	☐	
Store a queue of people waiting in line	☐	☐	

1.2 Exercise 1.2: Dictionary Vocabulary (7 points)

Match each term with its correct definition by writing the letter in the blank:

Term	Match	Definition
Key	---	A. The data you're storing (can be any type)
Value	---	B. Error when accessing a key that doesn't exist
Key-Value Pair	---	C. The unique label used to look up data
.get()	---	D. One entry in a dictionary (key + its value)
KeyError	---	E. Safe way to access a value with a default
.keys()	---	F. Returns all the labels in a dictionary
.items()	---	G. Returns all pairs as tuples

2 Part 2: Dictionary Creation and Access (18 points)

Time estimate: 25 minutes

2.1 Exercise 2.1: Code Tracing - Predict the Output (10 points)

What will each code snippet print? Trace carefully and write your prediction:

Snippet 1:

```
1 cities = {"Karachi": 15000000, "Lahore": 12000000, "Islamabad": 1100000}
2 print(cities["Lahore"])
```

Output: _____

Snippet 2:

```
1 fruits = {"apple": 5, "banana": 3}
2 fruits["orange"] = 7
3 fruits["apple"] = 10
4 print(fruits)
```

Output: _____

Snippet 3:

```
1 scores = {"Ali": 85, "Sara": 92}
2 print(scores.get("Hassan", 0))
3 print(scores.get("Ali", 0))
```

Output Line 1: _____

Output Line 2: _____

Snippet 4:

```
1 inventory = {"pens": 10, "notebooks": 5, "erasers": 20}
2 print(len(inventory))
3 print("pencils" in inventory)
```

Output Line 1: _____

Output Line 2: _____

Snippet 5:

```
1 grades = {"Math": 90, "Science": 85, "English": 88}
2 for subject in grades.keys():
3     print(subject, end=" ")
```

Output: _____

2.2 Exercise 2.2: Code Completion - Building a Dictionary (8 points)

Open [week11_ex2.2.py](#) and complete this program:

```
1 # Student Grade Manager
2 # Create an empty dictionary for student grades
3 student_grades = -----
4
5 # Add three students with their grades
6 student_grades["Fatima"] = -----
7 student_grades[-----] = 78
8 student_grades["Zainab"] = 95
9
10 # Print all student names
11 print("Students enrolled:")
12 for name in student_grades.-----:
13     print(f" - {name}")
14
15 # Check if "Ahmed" is in the class
16 if ----- in student_grades:
17     print(f"Ahmed's grade: {student_grades['Ahmed']}")
18 else:
19     print("Ahmed is not enrolled yet")
20
21 # Calculate class average
22 total = 0
```

```

23 for grade in student_grades:
24     total += grade
25 average = total / 
26 print(f"Class average: {average:.1f}")

```

3 Part 3: The Frequency Counter Pattern (20 points)

Time estimate: 25 minutes

3.1 Exercise 3.1: Understanding the Pattern (8 points)

The frequency counter is one of the most useful dictionary patterns. Study this code, then answer the questions:

```

1 # Count letter frequencies in a word
2 word = "mississippi"
3 letter_count = {}
4
5 for letter in word:
6     if letter in letter_count:
7         letter_count[letter] += 1
8     else:
9         letter_count[letter] = 1
10
11 print(letter_count)

```

Questions:

1. After processing just the first 4 letters ("miss"), what would `letter_count` contain?

2. Why do we check `if letter in letter_count` before incrementing?

3. What would happen if we removed the `else` branch and the letter wasn't in the dictionary yet?

4. Write the final output of `letter_count` for "mississippi":

3.2 Exercise 3.2: Apply the Pattern (12 points)

Open [week11_ex3.2.py](#) and complete these frequency counting exercises:

```

1 # Exercise A: Count word frequencies in a sentence
2 sentence = "the cat sat on the mat the cat was happy"
3 words = sentence.split() # Splits into list of words
4 word_count = {}
5
6 for word in words:
7     # Use the frequency counter pattern
8     if _____ in word_count:
9         word_count[_____] += 1

```

```

10     else:
11         word_count[_____] = 1
12
13 print("Word frequencies:")
14 print(word_count)
15
16 # Exercise B: Count number frequencies using .get() method
17 numbers = [1, 2, 1, 3, 2, 1, 4, 2, 1, 5]
18 num_count = {}
19
20 for num in numbers:
21     # Shorter way using .get()
22     num_count[num] = num_count._____(num, 0) + 1
23
24 print("\nNumber frequencies:")
25 print(num_count)
26
27 # Exercise C: Find the most common element
28 most_common = None
29 highest_count = 0
30
31 for item, count in num_count._____:
32     if count > highest_count:
33         highest_count = _____
34         most_common = _____
35
36 print(f"\nMost common number: {most_common} (appears {highest_count} times)"
    )

```

4 Part 4: Debugging Dictionary Code (15 points)

Time estimate: 20 minutes

Each snippet has bugs related to dictionaries. Find and fix ALL errors:

4.1 Snippet 1: The KeyError Bug (5 points)

```

1 # This code tries to look up player scores
2 # But it crashes when a player isn't found!
3
4 player_scores = {"Ali": 100, "Sara": 150, "Ahmed": 120}
5
6 # Get score for a player
7 player_name = "Hassan"
8 score = player_scores[player_name] # BUG: KeyError!
9 print(f"{player_name}'s score: {score}")

```

What's wrong? _____

Fixed version:

```

1 # Write the corrected code here (use .get() or check with 'in'):

```

4.2 Snippet 2: The Wrong Update (5 points)

```

1 # This code tries to count votes for favorite subjects
2 # But something is wrong with the counting!
3
4 votes = {}
5 subjects = ["Math", "Science", "Math", "English", "Math", "Science"]
6
7 for subject in subjects:
8     votes[subject] = 1 # BUG HERE!
9
10 print(votes)
11 # Expected: {'Math': 3, 'Science': 2, 'English': 1}
12 # Actual: {'Math': 1, 'Science': 1, 'English': 1}

```

What's wrong? _____

Fixed version:

```

1 # Write the corrected code here:

```

Save your corrected code as *week11_ex4_2_fixed.py*

4.3 Snippet 3: Looping Through Items (5 points)

```

1 # This code tries to print all contacts with their numbers
2 # But the loop isn't working correctly!
3
4 contacts = {"Mom": "0300-1111111", "Dad": "0300-2222222", "School": "042-3333333"}
5
6 for contact in contacts:
7     print(f"{contact}: {contact}") # BUG: Prints key twice!
8
9 # Expected output:
10 # Mom: 0300-1111111
11 # Dad: 0300-2222222
12 # School: 042-3333333

```

What's wrong? _____

Fixed version (show TWO different ways to fix this):

Way 1 - Using `.items()`:

Way 2 - Using key to access value:

5 Part 5: Full Program - Inventory Manager (22 points)

Time estimate: 35 minutes

5.1 Exercise 5.1: Build an Inventory System (22 points)

Create a program called `inventory_manager.py` that manages a small shop's inventory:

Program Requirements

Your program should:

- Start with an empty inventory dictionary
- Provide a menu with options: Add item, Update quantity, Check item, Show all, Exit
- Use a while loop to keep the program running until user exits
- Handle cases where items don't exist (use `.get()` or `in`)
- Display the inventory nicely formatted
- Validate that quantities are positive numbers

Example Run:

```
=== Inventory Manager ===
```

```
1. Add new item
2. Update item quantity
3. Check item stock
4. Show all inventory
5. Exit
```

```
Choose option (1-5): 1
Enter item name: Apples
Enter quantity: 50
Added: Apples (50 units)
```

```
Choose option (1-5): 1
Enter item name: Bananas
Enter quantity: 30
Added: Bananas (30 units)
```

```
Choose option (1-5): 3
Enter item to check: Oranges
Oranges not found in inventory.
```

```
Choose option (1-5): 3
Enter item to check: Apples
Apples: 50 units in stock
```

```
Choose option (1-5): 2
Enter item to update: Apples
Current stock: 50
Enter new quantity: 45
Updated: Apples now has 45 units
```

```
Choose option (1-5): 4
=== Current Inventory ===
  Apples: 45 units
  Bananas: 30 units
Total items tracked: 2
```

```
Choose option (1-5): 5
```

Thank you for using Inventory Manager!

Starter Structure:

```

1  # inventory_manager.py
2  # Name: -----
3  # Date: -----
4
5  print("=== Inventory Manager ===")
6
7  # Create empty inventory dictionary
8  inventory = {}
9
10 # Main program loop
11 running = True
12 while running:
13     # Display menu
14     print("\n1. Add new item")
15     print("2. Update item quantity")
16     print("3. Check item stock")
17     print("4. Show all inventory")
18     print("5. Exit")
19
20     choice = input("\nChoose option (1-5): ")
21
22     if choice == "1":
23         # Add new item
24         name = input("Enter item name: ")
25         quantity = int(input("Enter quantity: "))
26         # Add to inventory dictionary
27         -----
28         print(f"Added: {name} ({quantity} units)")
29
30     elif choice == "2":
31         # Update existing item
32         name = input("Enter item to update: ")
33         if ----- in inventory:
34             print(f"Current stock: {inventory[name]}")
35             new_qty = int(input("Enter new quantity: "))
36             -----
37             print(f"Updated: {name} now has {new_qty} units")
38         else:
39             print(f"{name} not found in inventory.")
40
41     elif choice == "3":
42         # Check item stock
43         name = input("Enter item to check: ")
44         # Use .get() to safely access
45         stock = inventory.get(-----, -----)
46         if stock is None:
47             print(f"{name} not found in inventory.")
48         else:
49             print(f"{name}: {stock} units in stock")
50
51     elif choice == "4":
52         # Show all inventory
53         if len(inventory) == 0:
54             print("Inventory is empty!")
55         else:
56             print("=== Current Inventory ===")
57             for item, quantity in inventory.-----():

```



```

58         print(f" {item}: {quantity} units")
59     print(f"Total items tracked: {_____}")
60
61     elif choice == "5":
62         # Exit
63         running = _____
64         print("Thank you for using Inventory Manager!")
65
66     else:
67         print("Invalid option. Please choose 1-5.")

```

Testing Checklist:

- ☐ Program runs without errors
- ☐ Can add new items to inventory
- ☐ Can update existing items
- ☐ Handles items that don't exist gracefully (no crashes)
- ☐ Shows all inventory correctly
- ☐ Exit option works properly
- ☐ Menu keeps appearing until exit is chosen

6 Part 6: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

- What is the main advantage of using a dictionary over a list when you need to look up information by name?

- Explain in your own words what the frequency counter pattern does and give one real-world example where you might use it.

- Why is `dict.get(key, default)` safer than `dict[key]`? When would you use each one?

7 Part 7: Bonus Challenges (Optional - 10 extra points)

Only attempt if you've finished everything else!

7.1 Challenge 1: Dictionary of Lists (5 points)

Create a program that groups students by their favorite subject. Use a dictionary where keys are subjects and values are lists of student names.

Open [week11_bonus1.py](#):

```

1 # Group students by favorite subject
2
3 students = [
4     ("Ali", "Math"),
5     ("Sara", "Science"),
6     ("Ahmed", "Math"),
7     ("Fatima", "English"),
8     ("Hassan", "Science"),
9     ("Zainab", "Math")
10 ]
11
12 # Create dictionary: subject -> list of students
13 subject_groups = {}
14
15 for name, subject in students:
16     # Your code here - add each student to the right subject list
17     # Hint: Check if subject exists, if not create empty list first
18     pass
19
20 # Print results
21 print("Students by Subject:")
22 for subject, names in subject_groups.items():
23     print(f"    {subject}: {names}")

```

7.2 Challenge 2: Two-Way Dictionary (5 points)

Create a program that can translate words both ways (English to Urdu AND Urdu to English) using two dictionaries.

Open [week11_bonus2.py](#):

```

1 # Two-way translator
2
3 # English to Urdu
4 eng_to_urdu = {
5     "hello": "salam",
6     "goodbye": "khuda hafiz",
7     "thank you": "shukriya",
8     "water": "pani",
9     "food": "khana"
10 }
11
12 # Create reverse dictionary (Urdu to English)
13 urdu_to_eng = {}
14 for eng, urdu in eng_to_urdu.items():
15     # Your code here
16     pass
17
18 # Translation function
19 def translate(word):
20     # Check English dictionary first, then Urdu
21     # Return translation or "Word not found"
22     pass

```

```

23
24 # Test your translator
25 print(translate("hello"))      # Should print: salam
26 print(translate("shukriya"))  # Should print: thank you
27 print(translate("computer"))  # Should print: Word not found

```

8 Debugging Discoveries (5 bonus points)

Growth Mindset: Errors are learning opportunities!

Did you encounter any interesting errors while completing this homework? Share one error and how you fixed it:

My Debugging Story

The Error I Got: _____

What I Tried First: _____

How I Fixed It: _____

What I Learned: _____

No errors? Share a tricky part you figured out instead!

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-6)
- ☐ Saved each code exercise in its own file:
 - week11_ex2.2.py
 - week11_ex3.2.py
 - week11_ex4.2_fixed.py
 - inventory_manager.py
- ☐ Created a folder named `week11_hw_yourname` with all files
- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Verified dictionaries use correct syntax (`{key: value}`)
- ☐ Checked that you handle missing keys safely (no `KeyError` crashes)
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)

Module 2 Concepts Check:

Which dictionary method was most useful? _____

What's the difference between `.keys()` and `.items()`? _____

Time Tracking:

How long did this homework take you? _____ hours

Parent/Guardian Signature: _____

(Confirming student completed work independently)

Great job mastering dictionaries and key-value pairs!

Next week: Functions - organizing your code into reusable building blocks!